# CS 519 Project 3: Part A Report

Saroop Makhija

## 1 Overview

The Linux scheduler is split between generic scheduler core code and per-class policies. CFS is not a standalone scheduler loop; it is one scheduling class plugged into a generic framework. The generic code in `kernel/sched/core.c` decides when scheduling happens, locks the current CPU's runqueue, updates accounting, and calls class callbacks. The CFS policy lives in `kernel/sched/fair.c` and serves `SCHED_NORMAL`, `SCHED_BATCH`, and `SCHED_IDLE`. Data structures live in `kernel/sched/sched.h`; nice and priority definitions are in `include/linux/sched/prio.h`; statistics paths run through `cputime.c`, `stats.h`, `fs/proc/array.c`, and the cgroup files.

## 2 Answers

### 2.1 How are tasks added to a per-CPU runqueue, and which file maintains this information?

Per-CPU runqueue state is struct rq in sched.h, instantiated by `DEFINE_PER_CPU_SHARED_ALIGNED` in `core.c`. A sleeping task enters through `try_to_wake_up()`; a forked task enters through `wake_up_new_task()`. Both end at `activate_task()`, which calls `enqueue_task()` and marks `p->on_rq`. `enqueue_task()` does the common bookkeeping (rq clock, sched_info, PSI, uclamp, core-scheduling) and dispatches to the class callback, which for CFS is `enqueue_task_fair()`. That walks sched entities and calls `enqueue_entity()`, which normalizes vruntime, inserts into the CFS rb-tree, updates PELT, bumps `nr_running`, and links the task on `rq->cfs_tasks` for load balancing.

### 2.2 How are tasks scheduled on a CPU, and which functions handle this?

`__schedule()` in `core.c` is the entry point. It locks the rq, updates clocks, deactivates the previous task if it is sleeping, and calls `pick_next_task()` in class order stop > dl > rt > fair > idle. For fair tasks this resolves to `pick_next_task_fair()`. CFS keeps runnable entities in `cfs_rq->tasks_timeline`, a cached red-black tree keyed by vruntime, and `pick_next_entity()` picks the leftmost (smallest vruntime), with limited override from the `next`, `last`, and `skip` buddy hints. The actual switch happens through `context_switch()`.

### 2.3 How are tasks descheduled or preempted from a CPU?

Voluntary deschedule happens when a task blocks or yields: `__schedule()` sees a non-running state, bumps `nvcsw`, calls `deactivate_task()` with `DEQUEUE_SLEEP`, and the CFS path `dequeue_task_fair()` removes the entity. If a task is being preempted but still runnable, it is not dequeued; `put_prev_task_fair()` just updates its runtime. Preemption itself is request-then-defer. `resched_curr()` sets `need_resched` (locally or via IPI), and the switch happens at the next

safe point. The triggers are wakeup preemption via `check_preempt_wakeup()`, tick preemption via `task_tick_fair()` → `check_preempt_tick()`, priority/class changes that hit `prio_changed()`, and explicit `sched_yield()`, which calls the class `yield_task()` callback.

### 2.4 How are runqueues rebalanced across CPUs?

Rebalancing is domain-based and lives in fair.c. `scheduler_tick()` calls `trigger_load_balance()`, which raises `SCHED_SOFTIRQ`; `run_rebalance_domains()` then walks the `sched_domain` hierarchy and calls `load_balance()` at each domain interval. Inside `load_balance()`: build an `lb_env`, gather stats with `update_sd_lb_stats()`, pick the busiest group and queue, choose a migration type in `calculate_imbalance()`, then `detach_tasks()` and `attach_tasks()`. `can_migrate_task()` rejects moves for affinity, per-CPU kthreads, currently running tasks, cache-hot locality, and throttled hierarchies; if passive balancing keeps failing, `active_load_balance_cpu_stop()` forces a migration. `sched_balance_newidle()` pulls work to a CPU going idle, NOHZ balancing handles tickless idle CPUs from a housekeeping CPU, and `select_task_rq_fair()` avoids later balancing by placing a waking task on a good CPU up front.

### 2.5 What data structures are used to manage runqueues?

The top-level `struct rq` owns the runqueue lock, `nr_running`, the `curr`/`idle`/`stop` pointers, per-class sub-runqueues, clock state, load-balancing fields, CPU capacity, uclamp state, and sched_domain hooks. Each class has its own sub-runqueue: `cfs_rq` (rb-tree by vruntime, buddy hints, PELT averages), `rt_rq` (per-priority array and bitmap), and `dl_rq` (rb-tree by deadline, bandwidth state). The per-task side mirrors this with `sched_entity` (CFS rb-node, load weight, `vruntime`, PELT average), `sched_rt_entity`, and `sched_dl_entity`, all embedded in `task_struct` along with a pointer to the active `sched_class`.

### 2.6 How does the scheduler convert application-specified nice values to priorities?

From `include/linux/sched/prio.h`:

```
NICE_TO_PRIO(nice) = nice + DEFAULT_PRIO
DEFAULT_PRIO = 120, MAX_RT_PRIO = 100
```

Nice $[-20, 19]$ maps to static priority $[100, 139]$ for fair-class tasks; smaller prio means higher priority. `set_user_nice()` validates bounds, sets `p->static_prio`, updates the CFS weight via `set_load_weight()` using the `sched_prio_to_weight[]` table, recomputes `p->prio`, and requeues. Each nice step is roughly 10% less weight, so CFS fairness is weight-based rather than driven by fixed timeslices. For RT and DL policies, nice is stored but does not drive scheduling.

### 2.7 How are per-task and per-process statistics collected by the scheduler?

Each class has an `update_curr` that maintains `sum_exec_runtime`. For CFS this computes `delta_exec` from `rq_clock_task()` and advances `vruntime` through `calc_delta_fair()`. User, system, and guest time are updated in `cputime.c`; voluntary and involuntary context switches (`nvcsw`, `nivcsw`) are bumped in `__schedule()`.

With `CONFIG_SCHEDSTATS`, `sched_statistics` tracks wait, sleep, and block time, max slice, migrations, and wakeup counters via the enqueue/dequeue path. With `CONFIG_SCHED_INFO`, `run_delay` and enqueue-arrival-depart timestamps are updated in `stats.h`. At the process level,

signal_struct holds group totals; __exit_signal() folds per-thread totals into it on exit, and thread_group_cputime() merges dead and live thread totals for queries.

### 2.8 How are these statistics exposed to userspace?

Mostly procfs. /proc/<pid>/stat and /status expose CPU times, faults, priority, nice, thread count, and voluntary/nonvoluntary context switches (fs/proc/array.c). /proc/<pid>/sched (under CONFIG_SCHED_DEBUG) prints scheduler internals through proc_sched_show_task(), and /proc/<pid>/schedstat (under CONFIG_SCHED_INFO) exports sum_exec_runtime, run_delay, and pcount. System-wide, /proc/stat exposes per-CPU cpustat buckets, and /proc/schedstat reports balancing and delay metrics. Cgroup v2 cpu.stat prints usage_usec, user_usec, and system_usec. Tracepoints like sched_switch and sched_wakeup are consumed via ftrace, perf, and eBPF.

## 3 CFS Idleness Mismatch and Underutilization

CFS can miss application idleness or leave cores underutilized in several cases. Blocking-dominated apps spend time in I/O, lock, or futex waits, so threads are not runnable and CFS has nothing to schedule. Cgroup CFS bandwidth throttling causes the same effect from quota exhaustion, and affinity or cpuset restrictions prevent idle CPUs outside the allowed mask from being used at all. A second cluster comes from estimation lag. PELT signals are decayed and averaged, so bursty workloads get misestimated and that affects placement and frequency. Cache-hot and locality heuristics make the balancer avoid migrations to protect locality, which delays load spread. NOHZ adds similar lag.The last cluster is structural. If an app has one runnable thread and many idle CPUs, CFS cannot parallelize the work; this shows up in single-threaded phases, serial bottlenecks, and dependency chains. External bottlenecks (storage, network, memory bandwidth, faults, interrupts) are the same problem in a different form: CPU idleness is a symptom of waiting outside the scheduler's runnable-task model.